DJANGO COMPLETE NOTES

Part 1: Django Installation — Complete Notes

1. How Do I Get Started with Django?

Django শেখা শুরু করার সাধারণ ধাপগুলো হলো:

1. Install Python.
2. Create a virtual environment.
3. Activate the virtual environment.
4. Install Django.
5. Create a Django project.
6. Run database migrations.
7. Run the development server.
8. Follow the Django tutorial.
9. Learn models, views, templates, forms, authentication and APIs.
10. Learn deployment.

2. Django Prerequisites

Django একটি Python web framework। Django development শুরু করার জন্য সাধারণত প্রয়োজন:

- Python
- Virtual environment
- Django
- SQLite database

Django শেখার জন্য শুরুতেই Apache, Nginx অথবা আলাদা database server install করা বাধ্যতামূলক নয়।
Django-তে development-এর জন্য নিজস্ব lightweight server রয়েছে।
Development server চালাতে:

```
python manage.py runserver
```

3. Which Python Versions Does Django Support?

Django-এর বিভিন্ন version-এর Python compatibility আলাদা। নিচের version mapping মূল source notes থেকে নেওয়া হয়েছে; installation-এর আগে সংশ্লিষ্ট Django release-এর official compatibility documentation যাচাই করবে।

Django Version	Python Versions

	in Source Notes
Django 4.2	Python 3.8, 3.9, 3.10, 3.11, 3.12
Django 5.0	Python 3.10, 3.11, 3.12
Django 5.1	Python 3.10, 3.11, 3.12, 3.13
Django 5.2	Python 3.10, 3.11, 3.12, 3.13, 3.14
Django 6.0	Python 3.12, 3.13, 3.14

কোন Python version ব্যবহার করবে, তা নির্ধারণের সময় বিবেচনা করবে:

- Django version
- Supported Python version
- Third-party package compatibility

Official Python download: https://www.python.org/downloads/

Python version check:

```
python --version
```

Windows-এ বিকল্প:

```
py --version
```

4. Stable Version vs Development Version

Stable Django: সাধারণ development এবং production project-এর জন্য উপযুক্ত।

Development Django: Django-এর নতুন feature পরীক্ষা করা বা Django source code-এ contribution করার কাজে ব্যবহৃত হয়।

সাধারণভাবে beginner হলে stable Django release ব্যবহার করবে।

5. What Is a Virtual Environment?

Virtual environment হলো একটি isolated Python environment। এটি প্রতিটি project-এর package ও dependency আলাদা রাখে।

উদাহরণ:

```
Project A
└── venv
    └── Django version A

Project B
└── venv
    └── Django version B
```

এতে একটি project-এর package update করলে অন্য project-এর dependency অপ্রয়োজনীয়ভাবে পরিবর্তিত হওয়ার ঝুঁকি কমে।

6. Create a Virtual Environment

Project directory তৈরি করো:

```
mkdir myproject
cd myproject
```

Virtual environment তৈরি করো:

```
python -m venv venv
```

Command-এর অর্থ:

- `python` : Python interpreter চালায়।
- `-m venv` : Python-এর venv module ব্যবহার করে।
- শেষের `venv` : environment-এর directory name।

7. Activate the Virtual Environment

Windows CMD:

```
venv\Scripts\activate
```

Windows PowerShell:

```
.\venv\Scripts\Activate.ps1
```

Linux / macOS:

```
source venv/bin/activate
```

Activation-এর পর terminal-এ সাধারণত `(venv)` দেখা যায়।

8. Install Django

Virtual environment activate করার পর:

```
python -m pip install Django
```

Django version check:

```
python -m django --version
```

অথবা:

```
django-admin --version
```

9. What Is pip?

`pip` হলো Python package installer। এটি Python package install, update এবং manage করতে ব্যবহৃত হয়।

Example:

```
python -m pip install Django
```

`python -m pip` ব্যবহার করলে নির্বাচিত Python interpreter-এর সঙ্গে সংশ্লিষ্ট pip চালানো সহজ হয়।

10. Django Database Support

Django-এর supported relational database backends-এর মধ্যে রয়েছে:

- PostgreSQL
- MariaDB
- MySQL
- SQLite
- Oracle

11. What Is SQLite?

SQLite একটি lightweight database। সাধারণত এটি একটি file-এর মধ্যে database data সংরক্ষণ করে।

```
db.sqlite3
```

SQLite উপযোগী:

- Learning

- Testing
- Small projects
- Prototyping

SQLite ব্যবহার করতে আলাদা database server প্রয়োজন হয় না।

12. Which Database Should I Use?

একটি সাধারণ learning setup:

```
Learning        → SQLite
Small Project   → SQLite or PostgreSQL
Production      → Requirements অনুযায়ী database
```

PostgreSQL অনেক production application-এর জন্য একটি ভালো পছন্দ। তবে final database নির্বাচন application-এর requirement অনুযায়ী করতে হবে।

13. Installing Database Drivers

SQLite ছাড়া অন্য database ব্যবহার করলে উপযুক্ত Python driver প্রয়োজন হতে পারে।

PostgreSQL:

```
python -m pip install "psycopg[binary]"
```

MySQL / MariaDB:

```
python -m pip install mysqlclient
```

Oracle:

```
python -m pip install oracledb
```

Driver installation-এর আগে সংশ্লিষ্ট package-এর official instructions এবং system compatibility দেখে নেবে।

14. Django Database Configuration

Django database configuration থাকে `settings.py` ফাইলে।

Default SQLite configuration:

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}
```

15. What Is ENGINE?

`ENGINE` setting Django-কে জানায় কোন database backend ব্যবহার করতে হবে।

SQLite:

```
"ENGINE": "django.db.backends.sqlite3"
```

PostgreSQL:

```
"ENGINE": "django.db.backends.postgresql"
```

MySQL:

```
"ENGINE": "django.db.backends.mysql"
```

Oracle:

```
"ENGINE": "django.db.backends.oracle"
```

16. What Is NAME?

SQLite-এর ক্ষেত্রে:

```
"NAME": BASE_DIR / "db.sqlite3"
```

এটি project-এর `BASE_DIR` directory-তে `db.sqlite3` file ব্যবহার করতে বলে। প্রয়োজন হলে Django migration-এর সময় file তৈরি করতে পারে।

17. Configuration for PostgreSQL

উদাহরণ:

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": "mydatabase",
        "USER": "myuser",
        "PASSWORD": "mypassword",
        "HOST": "localhost",
        "PORT": "5432",
    }
}
```

Settings-এর অর্থ:

- `ENGINE` : Database backend
- `NAME` : Database name
- `USER` : Database username
- `PASSWORD` : Database password

- `HOST` : Database server address
- `PORT` : Database server port

Security note: Production project-এ database password source code-এ hardcode না করে environment variable বা উপযুক্ত secret management ব্যবস্থা ব্যবহার করবে।

18. Creating a Database

PostgreSQL বা MySQL-এর মতো database ব্যবহার করলে আগে database তৈরি করতে হতে পারে।

Example SQL:

```
CREATE DATABASE database_name;
```

এরপর Django-এর `settings.py` -তে database connection configure করবে।

19. What Is a Migration?

Django migration হলো models-এর পরিবর্তন database schema-তে প্রয়োগ করার ব্যবস্থা।

সাধারণ workflow:

```
Django Models
      ↓
makemigrations
      ↓
Migration Files
      ↓
migrate
      ↓
Database Schema
```

প্রধান commands:

```
python manage.py makemigrations
python manage.py migrate
```

- `makemigrations` : Model changes থেকে migration files তৈরি করে।
- `migrate` : Migration files database-এ প্রয়োগ করে।

20. Is Apache Required for Django?

না। Django শেখার বা local development-এর জন্য Apache বাধ্যতামূলক নয়।

শুরুতে যথেষ্ট:

```
Python
   +
Virtual Environment
   +
```

```
Django
   +
SQLite
```

21. Production Web Server

Production deployment-এ একটি সাধারণ architecture হতে পারে:

```
Client / Browser
       ↓
Nginx or Apache
       ↓
Gunicorn / uWSGI / ASGI Server
       ↓
Django Application
       ↓
Database
```

Deployment stack application-এর প্রয়োজন অনুযায়ী নির্বাচন করতে হয়।

22. What Is WSGI?

WSGI-এর পূর্ণরূপ:

Web Server Gateway Interface

এটি Python web application এবং WSGI-compatible web server-এর মধ্যে standard interface।

Django-এর `wsgi.py` file সাধারণত এমন হয়:

```
import os

from django.core.wsgi import get_wsgi_application

os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "config.settings",
)

application = get_wsgi_application()
```

এখানে:

- `DJANGO_SETTINGS_MODULE` : Django settings module নির্ধারণ করে।
- `get_wsgi_application()` : WSGI application তৈরি করে।
- `application` : WSGI server ব্যবহার করতে পারে এমন callable।

Important: WSGI একটি interface specification; uWSGI হলো server software-এর নাম। দুটি একই জিনিস নয়।

23. What Is ASGI?

ASGI-এর পূর্ণরূপ:

Asynchronous Server Gateway Interface

ASGI synchronous ও asynchronous application handling সমর্থন করতে পারে এবং WebSocket-এর মতো protocol-এর জন্যও উপযোগী।

Django-এর `asgi.py` file সাধারণত এমন হয়:

```
import os

from django.core.asgi import get_asgi_application

os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "config.settings",
)

application = get_asgi_application()
```

24. Apache + mod_wsgi

Django-কে Apache-এর সঙ্গে `mod_wsgi` ব্যবহার করেও deploy করা যায়।

`mod_wsgi`-এর দুটি পরিচিত mode:

1. Embedded Mode
2. Daemon Mode

Embedded Mode: Python application code Apache process-এর মধ্যে চলে।

Daemon Mode: আলাদা daemon process Django request handle করে।

সঠিক deployment configuration project-এর requirement ও infrastructure-এর ওপর নির্ভর করে।

25. Complete Django Installation Process

```
Step 1: Install Python
Step 2: Create Project Directory
Step 3: Create Virtual Environment
Step 4: Activate Virtual Environment
Step 5: Install Django
Step 6: Create Django Project
Step 7: Run Migrations
Step 8: Run Development Server
```

26. Practical Django Installation — Windows

Step 1: Create project directory

```
mkdir myproject
cd myproject
```

Step 2: Create virtual environment

```
python -m venv venv
```

Step 3: Activate virtual environment

```
venv\Scripts\activate
```

Step 4: Install Django

```
python -m pip install Django
```

Step 5: Check Django version

```
python -m django --version
```

Step 6: Create Django project

```
django-admin startproject config .
```

এখানে `.` গুরুত্বপূর্ণ। এটি বর্তমান directory-তে project structure তৈরি করতে বলে।

Project structure:

```
myproject/
│
├── manage.py
├── config/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
│
└── venv/
```

27. Run Migrations

```
python manage.py migrate
```

এটি Django-এর প্রয়োজনীয় default migrations database-এ প্রয়োগ করে।

28. Run the Development Server

```
python manage.py runserver
```

Browser-এ খোলো:

```
http://127.0.0.1:8000/
```

29. Complete Installation Commands

```
mkdir myproject
cd myproject

python -m venv venv

# Windows CMD
venv\Scripts\activate

python -m pip install Django
python -m django --version

django-admin startproject config .

python manage.py migrate
python manage.py runserver
```

Linux/macOS-এ virtual environment activate করার command:

```
source venv/bin/activate
```

30. Development vs Production

Feature	Development	Production
Server	Django `runserver`	Production WSGI/ASGI stack
Database	SQLite often convenient	Requirements অনুযায়ী database

Debug	Development-এর জন্য	`DEBUG=False`
Security	Local development setup	Strict security configuration
Performance	Testing	Production monitoring ও optimization

Important: Django-এর `runserver` production server হিসেবে ব্যবহার করা উচিত নয়।

31. Installing Django's Development Version

Django source code নিয়ে কাজ করতে চাইলে repository clone করা যায়:

```
git clone https://github.com/django/django.git
```

তারপর project-এর virtual environment activate করে:

```
python -m pip install -e django/
```

এখানে `-e` মানে editable installation।

Source code update করতে:

```
cd django
git pull
```

সাধারণ beginner-এর জন্য development version প্রয়োজন নেই।

32. Beginner Learning Path

```
Python
   ↓
Virtual Environment
   ↓
Stable Django
   ↓
```

```
SQLite
   ↓
Django Tutorial
   ↓
Models
   ↓
Views
   ↓
Templates
   ↓
Forms
   ↓
Authentication
   ↓
Django REST Framework
   ↓
API Development
   ↓
Deployment
```

33. Most Important Installation Commands

```
# Create virtual environment
python -m venv venv

# Activate on Windows
venv\Scripts\activate

# Activate on Linux/macOS
source venv/bin/activate

# Install Django
python -m pip install Django

# Check Django version
python -m django --version

# Create project
django-admin startproject config .

# Apply migrations
python manage.py migrate
```

```
# Start development server
python manage.py runserver
```

34. Installation Short Summary

Django একটি Python-based web framework।

Django শেখার জন্য সাধারণত প্রয়োজন:

- Python
- Virtual environment
- Django

SQLite হলো সহজ development database, কারণ এটি আলাদা server ছাড়াই ব্যবহার করা যায়। Production-এর জন্য PostgreSQL-সহ বিভিন্ন database ব্যবহার করা সম্ভব।

প্রথমে Django project তৈরি, migrations, models, views এবং templates শেখো। Deployment technology পরে শেখা যায়।

PART 2: DJANGO REQUEST LIFECYCLE — COMPLETE NOTES

1. What Is the Django Request Lifecycle?

Django Request Lifecycle হলো browser থেকে আসা একটি HTTP request Django application-এর মধ্যে process হয়ে কীভাবে HTTP response হিসেবে browser-এ ফিরে যায়, সেই সম্পূর্ণ প্রক্রিয়া।

সহজভাবে: user যখন browser-এ কোনো URL প্রবেশ করে, Django request গ্রহণ করে, middleware ও URL routing-এর মাধ্যমে সঠিক view চালায়, প্রয়োজন হলে database থেকে data নেয়, response তৈরি করে এবং client-এ পাঠায়।

Complete Flow

```
Browser / Client
       |
       | HTTP Request
       v
Web Server
       |
       v
WSGI / ASGI
       |
       v
Request Middleware
       |
```

```
        v
URL Resolver (urls.py)
        |
        v
View (views.py)
        |
        +------> Model / ORM ------> Database
        |                ^              |
        |                |              |
        |           Query Result <------+
        |
        +------> Template Engine (if needed)
        |
        v
HTTP Response
        |
        v
Response Middleware
        |
        v
WSGI / ASGI
        |
        v
Web Server
        |
        v
Browser / Client
```

Important: সব request-এর জন্য database query বা template rendering বাধ্যতামূলক নয়।

2. User Sends an HTTP Request

প্রথমে user browser-এ একটি URL প্রবেশ করে।

Example:

```
https://example.com/products/5/
```

Browser ওই URL-এর জন্য HTTP request তৈরি করে server-এ পাঠায়।

একটি request-এ থাকতে পারে:

- HTTP method
- URL path
- Headers

- Cookies
- Query parameters
- Request body

Common HTTP methods:

```
GET
POST
PUT
PATCH
DELETE
```

Example:

```
GET /products/5/ HTTP/1.1
Host: example.com
Cookie: sessionid=abc123
```

এখানে:

- `GET` : HTTP method
- `/products/5/` : URL path
- `Host` : Server hostname
- `Cookie` : Browser থেকে পাঠানো cookie

3. Web Server and WSGI / ASGI

Browser থেকে request প্রথমে web server-এর কাছে পৌঁছায়। এরপর উপযুক্ত server/interface configuration-এর মাধ্যমে request Django application-এ পৌঁছায়।

WSGI

WSGI = Web Server Gateway Interface

এটি Python web application-এর traditional synchronous interface।

Examples of WSGI servers:

- Gunicorn
- uWSGI

ASGI

ASGI = Asynchronous Server Gateway Interface

এটি synchronous ও asynchronous application handling সমর্থন করে এবং WebSocket-এর মতো protocol-ও সমর্থন করতে পারে।

Examples of ASGI servers:

- Daphne

- Uvicorn
- Hypercorn

WSGI vs ASGI

Feature	WSGI	ASGI
Main use	Traditional synchronous applications	Synchronous ও asynchronous applications
Async handling	Traditional interface-এ সীমিত	Asynchronous handling সমর্থন করে
WebSockets	Standard WSGI interface-এর অংশ নয়	সমর্থন করতে পারে
Examples	Gunicorn, uWSGI	Uvicorn, Daphne, Hypercorn

Important: ASGI ব্যবহার করলেই Django application-এর সব code automatically asynchronous হয়ে যায় না।

4. Django Middleware — Request Phase

Middleware হলো Django-এর request/response processing chain-এর একটি reusable component।

Middleware:

- View execute হওয়ার আগে request process করতে পারে।
- View থেকে response ফেরার পর response process করতে পারে।

- Security checks চালাতে পারে।
- Session ও authentication-related processing করতে পারে।
- Request processing বন্ধ করে সরাসরি response return করতে পারে।

Common Django middleware:

```
SecurityMiddleware
SessionMiddleware
CommonMiddleware
CsrfViewMiddleware
AuthenticationMiddleware
MessageMiddleware
```

Request Middleware Flow

```
Request
   |
   v
Middleware 1
   |
   v
Middleware 2
   |
   v
Middleware 3
   |
   v
URL Resolver
```

Middleware-এর কাজ:

1. Request information process করা।
2. Security checks সম্পন্ন করা।
3. Session information load করা।
4. Authentication-related information যোগ করা।
5. প্রযোজ্য ক্ষেত্রে CSRF protection পরিচালনা করা।
6. প্রয়োজন হলে request reject করে response ফেরত দেওয়া।

Example: কোনো middleware request reject করলে request view পর্যন্ত না-ও পৌঁছাতে পারে।

5. URL Resolver — urls.py

Middleware processing-এর পর Django requested URL-এর সঙ্গে configured URL patterns match করে। এই কাজটি URL resolver করে।

Example `urls.py`:

```
from django.urls import path
from .views import product_detail

urlpatterns = [
    path(
        "products/<int:id>/",
        product_detail,
        name="product_detail",
    ),
]
```

ধরা যাক browser request করেছে:

```
/products/5/
```

Django নিচের pattern match করবে:

```
path(
    "products/<int:id>/",
    product_detail,
    name="product_detail",
)
```

এখানে:

- `products/` : Static URL segment
- `<int:id>` : Integer URL parameter
- `5` : `id` -এর value
- `product_detail` : যে view execute হবে

Django view-কে `id=5` argument দিয়ে call করবে।

What If the URL Does Not Match?

কোনো URL pattern match না করলে Django সাধারণত `404 Not Found` response দেয়।

6. View Execution — views.py

View হলো Django application-এর component, যা request গ্রহণ করে, application logic execute করে এবং HTTP response return করে।

View-এর কাজ হতে পারে:

1. Request গ্রহণ করা।
2. URL থেকে পাওয়া parameter গ্রহণ করা।
3. প্রয়োজন হলে database থেকে data খুঁজে বের করা।

4. Application logic execute করা।
5. Template বা API response তৈরি করা।
6. HTTP response return করা।

Function-Based View (FBV)

```
from django.http import HttpResponse


def hello(request):
    return HttpResponse("Hello, World!")
```

এখানে `hello()` function request গ্রহণ করে এবং text response return করে।

Product Detail FBV

```
from django.shortcuts import render
from .models import Product


def product_detail(request, id):
    product = Product.objects.get(id=id)

    return render(
        request,
        "product.html",
        {"product": product},
    )
```

এই view:

1. Request গ্রহণ করে।
2. URL থেকে `id` গ্রহণ করে।
3. ORM দিয়ে product খোঁজে।
4. Product data template-এ পাঠায়।
5. Render করা response return করে।

Class-Based View (CBV)

Django-তে class দিয়েও view তৈরি করা যায়।

```
from django.views.generic import DetailView
from .models import Product


class ProductDetailView(DetailView):
```

```
    model = Product
    template_name = "product.html"
```

FBV function দিয়ে view logic তৈরি করে, আর CBV class ব্যবহার করে।

7. Model, ORM and Database

View-এর logic অনুযায়ী database থেকে data প্রয়োজন হলে Django ORM ব্যবহার করা যায়।

ORM = Object-Relational Mapper

Django ORM-এর মাধ্যমে Python code ব্যবহার করে database-এর data query, create, update ও delete করা যায়।

Example:

```
product = Product.objects.get(id=5)
```

এটি database-এ `id=5` -এর সঙ্গে মিলে এমন product খোঁজে।

ORM Flow

```
View
  |
  v
Django ORM
  |
  v
SQL Query
  |
  v
Database
  |
  v
Query Result
  |
  v
Python Object
  |
  v
View
```

ধরা যাক database-এ product-এর data রয়েছে:

id	name	price

5	Laptop	75000

Code:

```
product = Product.objects.get(id=5)

print(product.name)
print(product.price)
```

Output:

```
Laptop
75000
```

Important: get()

`Product.objects.get(id=5)` কোনো matching object না পেলে `Product.DoesNotExist` exception raise করে।

একাধিক object match করলে `Product.MultipleObjectsReturned` exception raise হতে পারে।

8. Template Rendering

যখন HTML page তৈরি করতে হয়, তখন Django template engine ব্যবহার করা যায়।

View থেকে `render()` function-এর মাধ্যমে template ও context data পাঠানো হয়।

Example:

```
from django.shortcuts import render
from .models import Product


def product_detail(request, id):
    product = Product.objects.get(id=id)

    return render(
        request,
        "product.html",
        {"product": product},
    )
```

`product.html`:

```
<h1>{{ product.name }}</h1>
<p>Price: {{ product.price }}</p>
```

Context data:

```
{"product": product}
```

Rendering Flow

```
Template + Context Data
          |
          v
Django Template Engine
          |
          v
Rendered HTML
```

Template engine `{{ product.name }}` এবং `{{ product.price }}`-এর মতো variable-কে context-এর data দিয়ে render করে।

Example output:

```
<h1>Laptop</h1>
<p>Price: 75000</p>
```

Important: `render()` template render করে একটি `HttpResponse` object return করে। এটি নিজে browser-এ network request পাঠায় না; response Django-এর response-handling process-এর মাধ্যমে client-এ পৌঁছায়।

9. HTTP Response Is Created

View-এর শেষে Django একটি HTTP response object return করে।

Common response classes:

Response Class	Use
`HttpResponse`	সাধারণ text বা HTML response
`JsonResponse`	JSON response, বিশেষ করে API-তে

`HttpResponseRedirect`	Redirect response
`StreamingHttpResponse`	Streaming response
`TemplateResponse`	Template-based response

Example 1: HttpResponse

```
from django.http import HttpResponse


def hello(request):
    return HttpResponse("Hello, World!")
```

Example 2: JsonResponse

```
from django.http import JsonResponse


def api(request):
    return JsonResponse({
        "name": "Faruk",
        "age": 25,
    })
```

Example JSON output:

```
{
    "name": "Faruk",
    "age": 25
}
```

Example 3: Redirect

```
from django.shortcuts import redirect
```

```
def my_view(request):
    return redirect("home")
```

এখানে `"home"` হলো URL pattern-এর name।

Django redirect response-এ `Location` header পাঠায়। এরপর browser নতুন URL-এ আরেকটি request করতে পারে।

10. Middleware — Response Phase

View থেকে response return হওয়ার পর response middleware chain-এর মধ্য দিয়ে ফিরে আসে।

```
Request
   |
   v
Middleware 1
   |
   v
Middleware 2
   |
   v
Middleware 3
   |
   v
View
   |
   v
Middleware 3
   |
   v
Middleware 2
   |
   v
Middleware 1
   |
   v
Response
```

Response middleware-এর কাজ হতে পারে:

- Response headers যোগ করা।
- Cookies set করা।
- Security-related processing করা।
- Response content বা status পরিবর্তন করা।

- Caching-related processing করা।

Middleware-এর response processing সাধারণত request processing-এর বিপরীত ক্রমে চলে।

11. Web Server Sends the Response to the Browser

Django response processing শেষ হলে response WSGI/ASGI এবং web server-এর মাধ্যমে browser-এ পৌঁছায়।

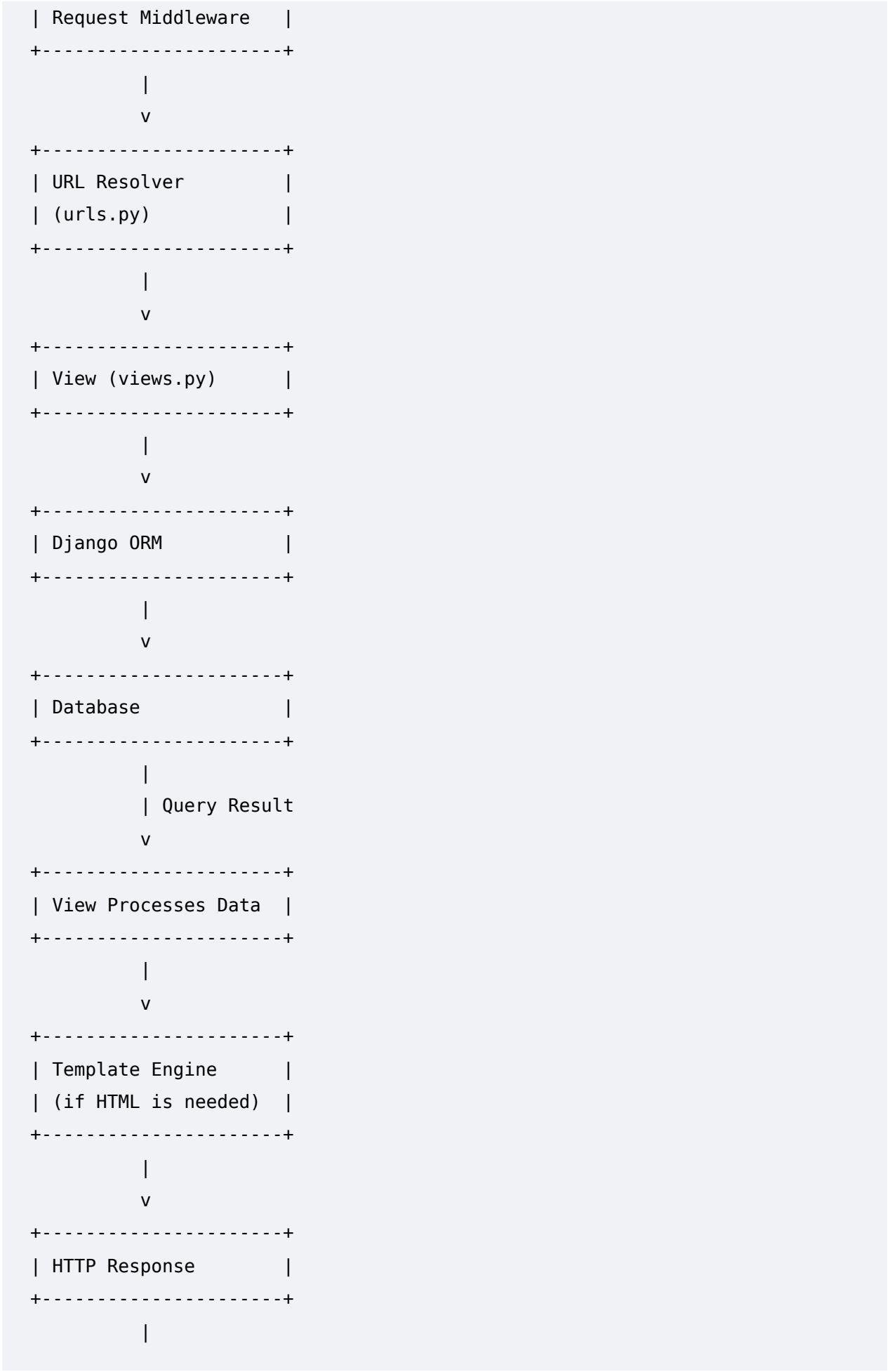
```
Django Application
        |
        v
     WSGI / ASGI
        |
        v
     Web Server
        |
        v
 Browser / Client
```

Browser response গ্রহণ করার পর:

- HTML হলে browser page render করে।
- JSON হলে browser বা frontend application JSON data ব্যবহার করতে পারে।
- Redirect response হলে browser নতুন URL-এ request করতে পারে।

12. Complete Django Request Lifecycle Diagram

```
+----------------------+
| Browser / Client     |
+----------------------+
          |
          | HTTP Request
          v
+----------------------+
| Web Server           |
+----------------------+
          |
          v
+----------------------+
| WSGI / ASGI          |
+----------------------+
          |
          v
+----------------------+
```


| Request Middleware |
| URL Resolver |
| (urls.py) |
| View (views.py) |
| Django ORM |
| Database |
| Query Result
| View Processes Data |
| Template Engine |
| (if HTML is needed) |
| HTTP Response |

```
          v
+----------------------+
| Response Middleware  |
+----------------------+
          |
          v
+----------------------+
| WSGI / ASGI          |
+----------------------+
          |
          v
+----------------------+
| Web Server           |
+----------------------+
          |
          v
+----------------------+
| Browser / Client     |
+----------------------+
```

এই diagram-এ database এবং template engine বোঝানোর জন্য ধারাবাহিকভাবে দেখানো হয়েছে। বাস্তবে database query ও template rendering view-এর প্রয়োজন অনুযায়ী হয়। সব request-এ দুটোই লাগবে না।

13. Real Example — Step by Step

ধরা যাক user browser-এ request করেছে:

```
GET /products/5/
```

আমরা একটি product detail page তৈরি করতে চাই।

Step 1 — Browser

Browser request পাঠায়:

```
GET /products/5/
```

Step 2 — Web Server and WSGI / ASGI

Web server request গ্রহণ করে Django application-এর কাছে পৌঁছে দেয়।

Step 3 — Request Middleware

Middleware প্রয়োজন অনুযায়ী request processing করে:

- Security checks
- Session processing
- Authentication processing

- প্রযোজ্য ক্ষেত্রে CSRF checks

Step 4 — URL Resolver

`urls.py`-এর pattern match হয়:

```
path("products/<int:id>/", product_detail)
```

Django `id=5` নির্ধারণ করে এবং `product_detail` view নির্বাচন করে।

Step 5 — View Execution

```
def product_detail(request, id):
    product = Product.objects.get(id=id)

    return render(
        request,
        "product.html",
        {"product": product},
    )
```

Step 6 — ORM and Database

```
Product.objects.get(id=5)
```

Django ORM database query চালায় এবং product object ফেরত দেয়।

Step 7 — Template Rendering

```
render(
    request,
    "product.html",
    {"product": product},
)
```

Django template ও product data ব্যবহার করে HTML তৈরি করে।

Step 8 — HTTP Response

View rendered response object return করে।

Step 9 — Response Middleware

Response middleware chain-এর মধ্য দিয়ে ফিরে আসে।

Step 10 — Browser

Browser final HTTP response গ্রহণ করে এবং HTML page প্রদর্শন করে।

14. Request Flow vs Response Flow

Request Flow

```
Browser
   ↓
Middleware 1
   ↓
Middleware 2
   ↓
Middleware 3
   ↓
URL Resolver
   ↓
View
```

Response Flow

```
View
   ↓
Middleware 3
   ↓
Middleware 2
   ↓
Middleware 1
   ↓
Browser
```

Shortcut: Request middleware chain-এর ভেতরে এগোয়; response সাধারণত middleware chain-এর বিপরীত ক্রমে ফিরে আসে।

15. Why Does Middleware Order Matter?

Middleware order গুরুত্বপূর্ণ, কারণ একটি middleware অন্য middleware-এর তৈরি করা information-এর ওপর নির্ভর করতে পারে।

উদাহরণ:

```
MIDDLEWARE = [
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
]
```

এখানে `SessionMiddleware` আগে রাখা হয়েছে, কারণ `AuthenticationMiddleware` session information ব্যবহার করে user authentication পরিচালনা করে।

তাই middleware-এর order random নয়। ভুল order হলে application error বা unexpected behavior দেখা দিতে পারে।

16. Important Concepts — Quick Revision

Concept	কাজ
Browser	HTTP request পাঠায় এবং response গ্রহণ করে
Web Server	HTTP traffic গ্রহণ করে এবং application-এর সঙ্গে যোগাযোগ করে
WSGI / ASGI	Server ও Python application-এর মধ্যে interface
Middleware	Request ও response process করে
URL Resolver	URL pattern match করে সঠিক view নির্ধারণ করে
View	Application logic

	execute করে এবং response return করে
Model	Database data-এর structure ও behavior নির্ধারণ করে
ORM	Python code দিয়ে database interaction সহজ করে
Database	Application data সংরক্ষণ করে
Template Engine	Template ও context data থেকে HTML তৈরি করে
HttpResponse	HTTP response object তৈরি করে

17. Interview Questions and Answers

Q1. What is the Django Request Lifecycle?

Answer: The Django Request Lifecycle is the complete process in which an HTTP request travels from the browser through the web server and WSGI/ASGI interface into Django, gets processed by middleware, URL routing, and a view, and finally returns an HTTP response to the client.

Q2. What does the URL Resolver do?

Answer: The URL resolver matches the requested URL against URL patterns and determines which view should handle the request.

Q3. What is Django Middleware?

Answer: Middleware is a reusable processing layer that can process HTTP requests before the view executes and HTTP responses after the view returns.

Q4. Can Middleware Block a Request?

Answer: Yes. Middleware can return an HTTP response directly without allowing the request to reach the view.

Q5. What does Django ORM do?

Answer: Django ORM provides a Python-based interface for interacting with relational databases, allowing developers to perform database operations using models and QuerySets.

Q6. What does render() do?

Answer: The `render()` function combines a template with context data, renders the template, and returns an `HttpResponse` containing the rendered content.

Q7. What is the difference between WSGI and ASGI?

Answer: WSGI is the traditional synchronous interface for Python web applications. ASGI supports asynchronous application handling and protocols such as WebSockets.

Q8. Why is Middleware Order Important?

Answer: Middleware order matters because some middleware depends on information provided by other middleware, and the processing order affects request and response behavior.

Q9. Does Every Django Request Need a Database Query?

Answer: No. A view can return a response without accessing the database.

Q10. Does Every Django Request Need Template Rendering?

Answer: No. Templates are commonly used for HTML responses, but APIs can return JSON using `JsonResponse` or other response classes without rendering an HTML

template.

18. Easy Memory Trick

এই flow-টি practice করো:

```
Browser
   ↓
Web Server
   ↓
WSGI / ASGI
   ↓
Middleware
   ↓
URL Resolver
   ↓
View
   ↓
Model / ORM / Database
   ↓
Template (if needed)
   ↓
HTTP Response
   ↓
Response Middleware
   ↓
Web Server
   ↓
Browser
```

Five Most Important Points

1. **Middleware:** Request ও response process করে।
2. **URL Resolver:** কোন view execute হবে তা নির্ধারণ করে।
3. **View:** মূল application logic execute করে।
4. **ORM:** Database interaction সহজ করে।
5. **Response:** Django থেকে server-এর মাধ্যমে browser-এ ফিরে যায়।

Interview-এর জন্য এক লাইনে

The Django Request Lifecycle starts when a client sends an HTTP request and ends when the client receives an HTTP response after Django processes the request through

the relevant middleware, URL resolver, view, and any required database or template operations.

FINAL REVISION CHECKLIST

Django Installation

- Python install ও version check করতে পারি।
- Virtual environment create ও activate করতে পারি।
- Django install করতে পারি।
- Django project create করতে পারি।
- `settings.py`-তে database configuration বুঝি।
- `makemigrations` ও `migrate`-এর পার্থক্য জানি।
- Development server চালাতে পারি।
- WSGI ও ASGI-এর basic ধারণা বুঝি।
- Development ও production environment-এর পার্থক্য জানি।

Django Request Lifecycle

- HTTP request-এর method, URL, headers ও body বুঝি।
- Web server এবং WSGI/ASGI-এর ভূমিকা বুঝি।
- Middleware-এর request ও response phase বুঝি।
- URL resolver কীভাবে view নির্বাচন করে জানি।
- FBV ও CBV-এর basic syntax জানি।
- ORM কীভাবে database-এর সঙ্গে কাজ করে বুঝি।
- Template rendering ও `render()` বুঝি।
- `HttpResponse`, `JsonResponse` ও redirect বুঝি।
- Request flow ও response flow ব্যাখ্যা করতে পারি।
- Interview-এর ১০টি প্রশ্নের উত্তর দিতে পারি।

END OF COMPLETE NOTES